



BOULAY · BEACH · RESORT

BBR

Booking Engine

Spécifications fonctionnelles & techniques

Moteur de réservation unifié multi-produits

Version 1.0 · Édition juin 2026

Document confidentiel — Produit & Technique

SOMMAIRE

Table des matières

Résumé exécutif	p. 4
§1 — Architecture du Booking Engine	p. 6
§2 — Flux fonctionnels (8 types de réservation)	p. 9
§3 — Wireframes du tunnel de réservation	p. 13
§4 — Modèle de données du Booking Engine	p. 16
§5 — API REST détaillées (Swagger)	p. 20
§6 — Règles métier (annulation, no-show, reprog.)	p. 26
§7 — Gestion des disponibilités	p. 28
§8 — Gestion des paiements	p. 30
§9 — Gestion des QR Codes	p. 33
§10 — Plan d'intégration PMS + OTA	p. 35
Annexe A — Espace client	p. 38
Annexe B — Dashboard interne	p. 39
Annexe C — Contrôle d'accès (scanner)	p. 40
Annexe D — KPIs et analytics	p. 41

Un seul moteur, toutes les ventes

Le **Booking Engine BBR** est le cœur transactionnel de la plateforme Revenue Engine. C'est lui qui transforme un visiteur intéressé en client payant — pour **n'importe quel produit BBr** : hébergement, beach club, activités nautiques, événements privés, séminaires corporate ou cartes de fidélité.

Objectifs mesurables

- **Taux de conversion tunnel $\geq 32\%$** (vs $\sim 12\%$ en moyenne hôtelière). Atteint grâce à un tunnel optimisé en 7 étapes, upsell/cross-sell intelligent et zéro friction de paiement.
- **Panier moyen +18%** grâce aux options additionnelles automatiques (Sunset, Brunch, Jet Ski, transfert, massage).
- **Zéro double-réservation** via verrouillage atomique d'inventary et hold panier 5 min.
- **Délai de confirmation < 4 sec** entre paiement validé et QR + ticket PDF + Email + WhatsApp reçus par le client.
- **Capacité $\geq 50\,000$ réservations/mois** sur architecture actuelle (palier 1 du plan de scalabilité).

Principes directeurs

- **Une seule source de vérité** — toutes les ventes (directes, OTA, manuelles, corporate) transitent par le même moteur. Pas de silo.
- **Mobile-first** — 73% des bookings BBr se font sur smartphone. Le tunnel est conçu pour fonctionner parfaitement sur un écran de 5 pouces.
- **Paiement local-first** — Wave, Orange Money, MTN Money et Moov Money intégrés en priorité (90% des transactions en Côte d'Ivoire). Cartes en complément.
- **Idempotent et auditable** — chaque réservation possède une Idempotency-Key, un audit trail complet et une référence humaine immuable (BBR-YYYYMMDD-XXXX).
- **Multi-langue** dès le lancement : Français (par défaut), Anglais, Espagnol (touristes UE/US qui viennent en CI).

Périmètre du document

Ce document définit le module **Booking Engine** tel qu'il s'inscrit dans la plateforme Revenue Engine. Il s'appuie sur le modèle de données PostgreSQL livré dans le document d'architecture (§5). Les choix de stack et de scalabilité y sont déjà arbitrés.

Architecture du Booking Engine

Le Booking Engine est organisé en **3 couches verticales** alimentées par 4 points d'entrée et déversant vers 4 services post-paiement. Le diagramme ci-dessous présente la vue d'ensemble.

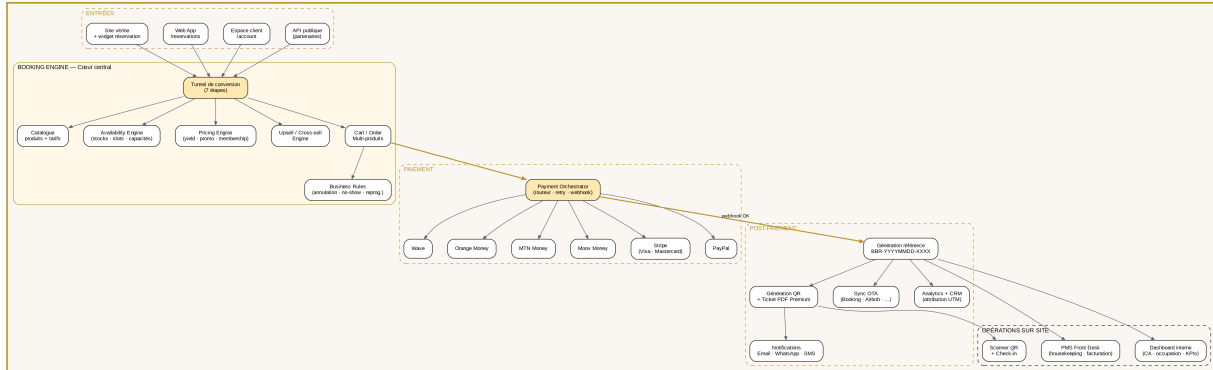


Schéma 1 — Architecture du Booking Engine.

Composants du cœur

Tunnel de conversion

Composant frontend / API qui orchestre les 7 étapes du parcours. État stocké côté client (Zustand) et côté serveur (session Redis 30 min) pour garantir la reprise en cas de fermeture d'onglet.

Catalogue & Pricing Engine

Le catalogue lit la table products + inventory. Le pricing engine applique en cascade : (1) prix de base, (2) overrides de saison, (3) règles de yield, (4) code promo, (5) tier membership. La fonction est **pure et déterministe** — même entrée = même prix, traçable.

Availability Engine

Garant absolu du *no-double-booking*. Utilise une transaction PostgreSQL SERIALIZABLE avec verrouillage ligne sur inventory au moment du checkout. Un **hold de 5 minutes** est posé pendant l'étape de paiement. À l'expiration, l'inventory est automatiquement libéré.

Upsell / Cross-sell Engine

Règles configurables côté admin : "Si produit X dans le panier ALORS suggérer Y, Z avec discount D". Algorithme complémentaire d'IA légère (XGBoost sur les co-achats des 12 derniers mois) pour optimiser au fil de l'eau.

Business Rules Engine

Règles d'annulation, no-show, reprogrammation, période de grâce. Définies en JSONB sur chaque produit pour flexibilité maximale (les chambres et le brunch n'ont pas les mêmes règles).

Payment Orchestrator

Façade unifiée devant 8 PSPs (Wave, Orange Money, MTN, Moov, Stripe Visa/MC, PayPal). Implémente le pattern **circuit breaker** : si un PSP est en panne, le routeur bascule automatiquement sur un fallback compatible.

Flux fonctionnels par type de réservation

Les 8 types de réservation partagent le même tunnel global, mais activent des règles spécifiques. Tableau récapitulatif :

Type	Capacité	Slots	Paielement	Annulation	QR
Chambre	1 nuit/chambre	checkin/out	Acompte 50% ou 100%	72h flexible	1 par séjour
Day Pass	250 / jour	all-day	100% obligatoire	24h avant	1 par billet
Sunset	120 / soir	16h-20h	100% obligatoire	24h avant	1 par billet
Brunch	200 / dimanche	11h-15h	100% obligatoire	24h avant	1 par billet
Activité	varie	créneaux 30-60 min	100% obligatoire	12h avant	1 par activité
Corporate	1 salle	demi-journée	Acompte 30% + solde	Sur devis	1 par groupe
Événement	1 espace	soirée	Acompte 50% + solde	Sur devis	1 organisateur + invités
Membership	n/a	carte annuelle	100% obligatoire	Non remboursable	1 carte numérique

2.1 — Hébergement (Suites + Chambres)

Trois catégories : **Chambre Exclusive**, **Suite Lagune**, **Suite Jardin**. Chaque produit possède une galerie photo, une description riche en markdown, un prix de référence par nuit, une capacité (adultes + enfants), une liste d'équipements, et une politique d'annulation spécifique. La disponibilité est calculée chambre par chambre (inventory atomique).

2.2 — Beach Club (Day Pass · Sunset · Brunch)

Trois offres quotidiennes avec quota partagé. Day Pass = 250 personnes/jour, Sunset = 120, Brunch = 200 (dimanches). Les tarifs distinguent adulte / enfant (3-12 ans) / bébé. Inclusions détaillées (accès piscine, transat, serviette, etc.). Le QR est le boarding pass délivré actuellement.

2.3 — Activités

7 activités : **Jet Ski**, **Paddle**, **Canoë**, **Padel**, **Quad**, **Buggy**, **Multisports**. Chacune avec son nombre d'équipements, ses créneaux horaires et sa durée. Réservation par **slot** de 30 ou 60 min. Si tous les jet skis sont pris à 10h, l'utilisateur voit automatiquement la prochaine plage disponible.

2.4 — Corporate

Réservation de salle (séminaire, conférence, team building, journée d'étude). Champs spécifiques : nombre de participants, options restauration (cocktail, déjeuner, dîner), options techniques (vidéoprojecteur, sonorisation, traduction simultanée). **Paielement en deux temps** : acompte 30% à la signature du devis + solde 7 jours avant l'événement.

2.5 — Événementiel

Mariages, anniversaires, soirées privées, concerts. **Demande de devis intégrée** au tunnel : le client renseigne ses besoins, un commercial reçoit la demande sur le dashboard et génère un devis. Une fois validé, conversion en réservation standard avec acompte 50%.

2.6 — Membership

Trois tiers : **Bronze** (50k XOF/an, 10% de réduction), **Or** (150k XOF/an, 20% + accès prioritaire), **Platine** (400k XOF/an, 30% + chambre offerte/an + transferts inclus). Paiement intégral à la souscription, renouvellement automatique via le PSP enregistré.

Upsell & Cross-sell — règles initiales

Produit dans le panier	Suggestions automatiques	Discount
Chambre	Sunset · Brunch · Transfert · Activité	10% si bundle ≥ 2
Day Pass	Jet Ski · Paddle · Massage · Déjeuner	15% sur l'activité
Sunset	Dîner Le Kaai · Transfert retour	Cocktail offert
Brunch	Day Pass (le matin) · Spa après-midi	Day Pass –30%
Corporate	Team building activités · Soirée gala	Sur devis
Événement	Hébergement invités · Brunch lendemain	Tarif groupe

Wireframes du tunnel de réservation

Le tunnel se compose de **7 étapes**. Objectif : chaque étape doit pouvoir être complétée en moins de **15 secondes** sur mobile. L'utilisateur peut *toujours revenir en arrière* sans perdre ses données (session Redis 30 min).

Vue d'ensemble — entonnoir de conversion

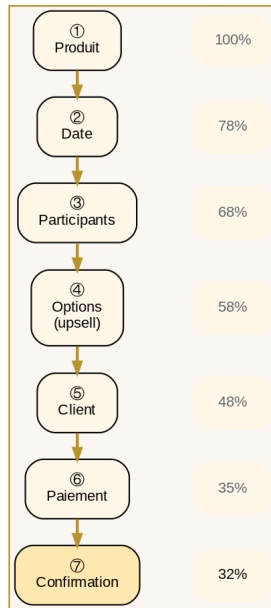


Schéma 2 — Tunnel à 7 étapes avec objectifs de conversion par palier (objectif global $\geq 32\%$ du visiteur au booking confirmé).

Wireframes des 7 étapes

Maquette filaire de chaque écran. Le rendu final s'appuiera sur le design system existant (Tailwind + shadcn/ui) avec la charte BBr (noir + or + cream).

ÉTAPE 1 — Choisissez votre expérience		
HÉBERGEMENT Suites + Chambres	BEACH CLUB Day Pass · Sunset · Brunch	ACTIVITÉS Jet ski · Quad · Padel · ...
CORPORATE Séminaires · Team building	ÉVÉNEMENTIEL Mariages · Soirées	MEMBERSHIP Carte fidélité

↓

ÉTAPE 2 — Quand ?		
Date d'arrivée []/[]/[]	Date de départ []/[]/[]	Calendrier (mois courant + suivant) • disponible ○ tarif majoré X complet

↓

ÉTAPE 3 — Combien de participants ?		
Adultes [-] 2 [+]	Enfants (3-12 ans) [-] 0 [+]	Bébés (gratuit) [-] 0 [+]
Total estimé : 145 000 XOF — 2 adultes en Suite Lagune, 2 nuits		

↓

ÉTAPE 4 — Ajoutez à votre séjour (upsell + cross-sell)		
☐ Transfert privé 30 000 XOF	☐ Brunch dominical 22 000 XOF / pers.	☐ Massage couple 45 000 XOF
☐ Sunset Experience 15 000 XOF / pers.	☐ Jet ski 30 min 35 000 XOF	☐ Padel court 1h 12 000 XOF

↓

ÉTAPE 5 — Vos informations	
Nom · Prénom []	Téléphone (WhatsApp) [+225 []]
Email []	Nationalité [Sénégalaise ▼]
☐ J'accepte les CGV — ☐ Je souhaite recevoir les offres BBr	

↓

ÉTAPE 6 — Paiement sécurisé		
☐ Acompte 50% 72 500 XOF Solde à payer avant arrivée	☒ Paiement intégral 145 000 XOF Annulation flexible jusqu'à 72h	☐ Paiement sur place (sur autorisation)
☐ Wave]	☐ Orange Money]	☐ MTN Money]
☐ Moov Money]	☐ Visa · Mastercard]	☐ PayPal]

↓

ÉTAPE 7 — Confirmé ✓	
Réservation BBR-20260712-A4F2	
- Ticket PDF Premium téléchargé - QR Code généré - Email + WhatsApp envoyés	Télécharger le ticket Ajouter au calendrier Voir mes réservations

Schéma 3 — Wireframes textuels des 7 écrans du tunnel.

Détails par étape

Étape 1 — Sélection produit

Grille 6 tuiles (3×2 sur desktop, 2×3 sur mobile). Chaque tuile affiche une photo full-bleed, un titre, une accroche d'une ligne. Si l'utilisateur arrive avec un UTM `utm_content=day_pass`, la tuile correspondante est **présélectionnée** et le tunnel saute directement à l'étape 2.

Étape 2 — Date

Daterangepicker double (arrivée + départ) avec calendrier couleur. Pour les offres mono-date (Day Pass, Sunset, Brunch, Activités), un seul champ. Les dates indisponibles sont grisées. Hover sur une date affiche le prix du jour.

Étape 3 — Participants

Steppers ± pour adultes / enfants / bébés. Validation en temps réel selon la capacité du produit (impossible de dépasser). Le total estimé s'affiche en direct.

Étape 4 — Options (upsell + cross-sell)

Liste de 4 à 8 propositions selon le produit. Format **opt-in explicite** (cases à cocher, jamais pré-cochées) pour conformité Dark Pattern. Chaque suggestion montre son prix et son économie si elle est en bundle.

Étape 5 — Client

Formulaire minimaliste : nom, prénom, téléphone WhatsApp, email, nationalité. Auto-complétion si l'utilisateur est *logged-in*. Acceptation CGV obligatoire. Consentement marketing opt-in séparé.

Étape 6 — Paiement

Choix du mode (acompte 50% / intégral / sur place si autorisé) puis du PSP. Les méthodes mobile money (Wave, OM, MTN, Moov) sont affichées **en premier** (90% du marché ivoirien). Stripe + PayPal en second.

Étape 7 — Confirmation

Écran de succès avec référence visible, téléchargement du ticket PDF, ajout au calendrier (.ics), liens vers l'espace client. Envoi automatique simultané d'un email + d'un WhatsApp avec le ticket en pièce jointe.

Modèle de données du Booking Engine

Le Booking Engine s'appuie sur le modèle central défini dans le document d'architecture (tables customers, products, inventory, reservations, payments, qr_codes) et ajoute **6 tables spécifiques** : holds panier, options additionnelles, devis, codes promo, refunds, reschedules.

État (machine à états) d'une réservation

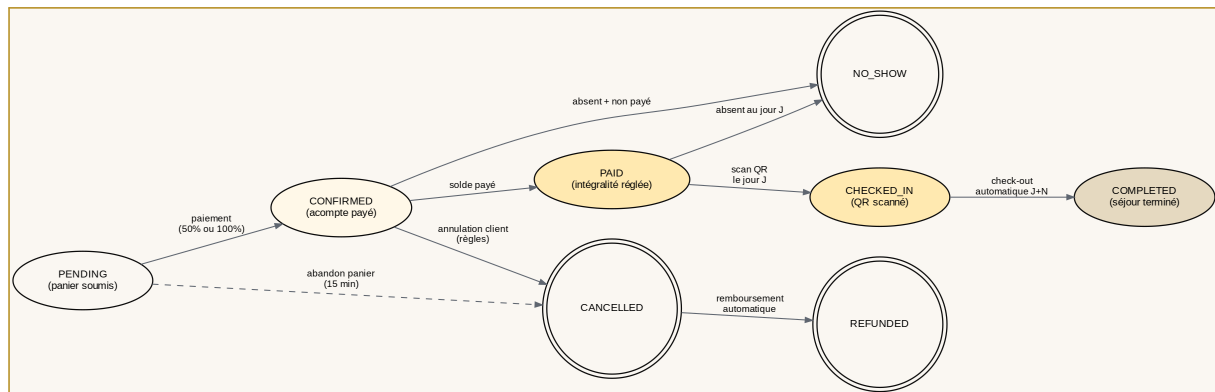


Schéma 4 — Machine à états d'une réservation.

DDL des 6 tables ajoutées

```
-- Extensions du schéma revenue_engine pour le Booking Engine

-- 14. Hold panier (5 min avant paiement)
CREATE TABLE booking_holds (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id  UUID NOT NULL,
  visitor_id  UUID,
  product_id  UUID NOT NULL REFERENCES products(id),
  date        DATE NOT NULL,
  slot        TEXT,
  quantity    INTEGER NOT NULL,
  options     JSONB NOT NULL DEFAULT '[]',
  quoted_amount NUMERIC(12,2) NOT NULL,
  expires_at  TIMESTAMPTZ NOT NULL,
  converted_to UUID REFERENCES reservations(id),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_hold_session ON booking_holds(session_id, expires_at);
CREATE INDEX idx_hold_expiry ON booking_holds(expires_at)
  WHERE converted_to IS NULL;

-- 15. Options additionnelles d'une réservation (upsell / cross-sell)
CREATE TABLE reservation_options (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reservation_id UUID NOT NULL REFERENCES reservations(id) ON DELETE CASCADE,
  option_code  TEXT NOT NULL,          -- 'transfer', 'sunset', 'massage', ...
  label_fr    TEXT NOT NULL,
  quantity    INTEGER NOT NULL DEFAULT 1,
  unit_price  NUMERIC(10,2) NOT NULL,
  total_price NUMERIC(10,2) NOT NULL,
  is_upsell   BOOLEAN NOT NULL DEFAULT false
);
CREATE INDEX idx_resopt_res ON reservation_options(reservation_id);

-- 16. Devis (Corporate / Événementiel)
CREATE TABLE quotes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quote_number TEXT UNIQUE NOT NULL,  -- 'BBR-Q-2026-0042'
  customer_id UUID NOT NULL REFERENCES customers(id),
  product_id  UUID REFERENCES products(id),
  target_date  DATE,
  participants INTEGER,
  requested_options JSONB NOT NULL DEFAULT '{}',
  estimated_total NUMERIC(12,2),
  final_total    NUMERIC(12,2),
  status         TEXT NOT NULL DEFAULT 'draft',
                -- draft, sent, accepted, rejected, expired, converted
  sent_at       TIMESTAMPTZ,
  accepted_at   TIMESTAMPTZ,
  converted_to  UUID REFERENCES reservations(id),
  commercial_id UUID REFERENCES users(id),
  notes         TEXT,
  pdf_url       TEXT,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at    TIMESTAMPTZ
);

-- 17. Codes promo / coupons
CREATE TABLE promo_codes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code        TEXT UNIQUE NOT NULL,
  discount_type TEXT NOT NULL,          -- 'percent' or 'amount'
  discount_value NUMERIC(10,2) NOT NULL,
  min_amount    NUMERIC(10,2),
  applies_to    module_type[],          -- modules concernés (null = tous)
  starts_at     TIMESTAMPTZ,
  ends_at       TIMESTAMPTZ,
  max_uses      INTEGER,
  used_count    INTEGER NOT NULL DEFAULT 0,
```

```

        campaign_id    UUID REFERENCES campaigns(id),
        status         TEXT NOT NULL DEFAULT 'active'
    );

-- 18. Refunds (note de crédit)
CREATE TABLE refunds (
    id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id        UUID NOT NULL REFERENCES payments(id),
    reservation_id    UUID NOT NULL REFERENCES reservations(id),
    amount            NUMERIC(12,2) NOT NULL,
    reason             TEXT NOT NULL,
                    -- 'client_cancel', 'no_show_grace',
                    -- 'overbooking', 'staff_error'

    requested_by      UUID REFERENCES users(id),
    approved_by       UUID REFERENCES users(id),
    provider_refund_ref TEXT,
    status            TEXT NOT NULL DEFAULT 'pending',
                    -- pending, processing, completed, failed
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at      TIMESTAMPTZ
);

-- 19. Reprogrammations
CREATE TABLE reschedules (
    id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reservation_id    UUID NOT NULL REFERENCES reservations(id),
    old_date          DATE NOT NULL,
    new_date          DATE NOT NULL,
    old_slot          TEXT,
    new_slot          TEXT,
    requested_by      UUID REFERENCES users(id),
    fee_applied       NUMERIC(10,2) NOT NULL DEFAULT 0,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Pourquoi une table booking_holds dédiée ?

Le hold est **éphémère** (TTL 5 min). Le stocker dans une table dédiée plutôt que dans reservations évite de polluer la table principale et facilite la purge automatique. Une réservation n'existe officiellement qu'après validation du paiement.

API REST détaillées (Swagger / OpenAPI 3.1)

Toutes les routes sont organisées en **4 namespaces** : publique (parcours de réservation), espace client (self-service authentifié), staff (back-office), webhooks (entrants PSP/OTA). Documentation Swagger générée automatiquement à `/api/docs`.

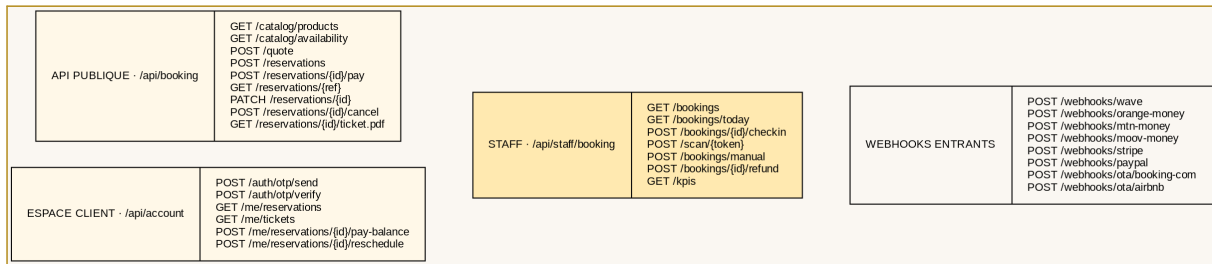


Schéma 5 — Cartographie des 4 namespaces d'API du Booking Engine.

5.1 — Endpoints publics (sans auth)

Méthode	URL	Description
GET	/api/booking/catalog/products	Liste catalogue (filtres : module, category, date)
GET	/api/booking/catalog/availability	Dispo par produit × date × slot (cache 30s)
POST	/api/booking/quote	Calcule le prix sans réserver (pour affichage panier)
POST	/api/booking/reservations	Crée la réservation + le hold de 5 min
POST	/api/booking/reservations/{id}/pay	Lance le paiement (retourne redirect_url ou OTP)
GET	/api/booking/reservations/{ref}	Récupère via la référence humaine (lookup public)
GET	/api/booking/reservations/{id}/ticket.pdf	Retourne le ticket PDF (token signé)

5.2 — Endpoints espace client (auth OTP)

Méthode	URL	Description
POST	/api/account/auth/otp/send	Envoie OTP par WhatsApp/SMS
POST	/api/account/auth/otp/verify	Vérifie OTP → JWT 7 jours
GET	/api/account/me/reservations	Toutes mes résa (pagination)
GET	/api/account/me/tickets	Tickets téléchargeables
POST	/api/account/me/reservations/{id}/pay-balance	Paiement du solde
POST	/api/account/me/reservations/{id}/reschedule	Reprogrammation (selon règles)
POST	/api/account/me/reservations/{id}/cancel	Annulation (selon règles)

5.3 — Endpoints staff

Méthode	URL	Description
GET	/api/staff/booking/bookings	Liste (filtres : statut, date, produit, canal)
GET	/api/staff/booking/bookings/today	Vue du jour pour la réception

Méthode	URL	Description
POST	/api/staff/booking/bookings/{id}/checkin	Check-in manuel (sans QR)
POST	/api/staff/scan/{token}	Scan QR + validation accès
POST	/api/staff/booking/bookings/manual	Création manuelle (téléphone, comptoir)
POST	/api/staff/booking/bookings/{id}/refund	Refund (validation hiérarchique)
GET	/api/staff/booking/kpis	KPIs temps réel (CA, occupation, no-show)

5.4 — Webhooks entrants (PSP & OTA)

Tous les webhooks sont **signés HMAC** (header X-Signature-256) et validés avant traitement. Idempotence par event_id stocké dans Redis 7 jours.

URL	Source	Événements
/api/booking/webhooks/wave	Wave	payment.completed · payment.failed
/api/booking/webhooks/orange-money	Orange Money	same
/api/booking/webhooks/mtn-money	MTN Money	same
/api/booking/webhooks/moov-money	Moov Money	same
/api/booking/webhooks/stripe	Stripe	payment_intent.succeeded · charge.refunded
/api/booking/webhooks/paypal	PayPal	PAYMENT.CAPTURE.COMPLETED · REFUNDED
/api/booking/webhooks/ota/booking-com	Booking.com	reservation.new · reservation.modified · canceled
/api/booking/webhooks/ota/airbnb	Airbnb	same

5.5 — Exemple : créer une réservation

POST /api/booking/reservations

Headers:

Content-Type: application/json

Idempotency-Key: 7c9a4f12-bf30-4b69-a1c2-09e4567a89bd

Accept-Language: fr-FR

Body:

```
{
  "product_id": "5e8d3c1a-...",
  "date_in": "2026-07-12",
  "date_out": "2026-07-14",
  "slot": null,
  "adults": 2,
  "children": 0,
  "options": [
    {"code": "transfer", "quantity": 1},
    {"code": "sunset", "quantity": 2}
  ],
  "customer": {
    "first_name": "Adama",
    "last_name": "Sangaré",
    "email": "adama@example.com",
    "phone": "+2250707070707",
    "nationality": "CI",
    "language": "fr"
  },
  "promo_code": "WELCOME10",
  "utm": {
    "source": "meta",
    "medium": "paid_social",
    "campaign": "summer_2026_daypass",
    "content": "carousel_v3"
  },
  "consent_marketing": true,
  "session_id": "9a2b...-fd45"
}
```

Response 201 Created:

```
{
  "id": "f3e7-...",
  "reference": "BBR-20260712-A4F2",
  "status": "PENDING",
  "amount_total": 145000,
  "amount_due": 145000,
  "currency": "XOF",
  "hold_expires_at": "2026-06-17T18:55:00Z",
  "payment_intent": {
    "client_secret": "pi_301...",
    "providers": ["wave", "orange_money", "mtn", "moov",
                  "stripe", "paypal"]
  }
}
```

5.6 — Exemple : lancer le paiement

POST /api/booking/reservations/{id}/pay

```
{
  "payment_method": "wave",
  "amount": 72500,
  "type": "deposit"
}
```

Response 200 OK:

```
{
  "payment_id": "8b1f-...",
  "redirect_url": "https://pay.wave.com/checkout/...",
  "expires_in": 300
}
```

5.7 — Exemple : scan QR (staff)

GET /api/staff/scan/{token}

Response 200 OK:

```
{
  "reservation": {
    "reference": "BBR-20260712-A4F2",
    "status": "PAID",
    "product": "Suite Lagune 2 nuits",
    "date_in": "2026-07-12",
    "date_out": "2026-07-14",
    "guest": {
      "name": "Adama Sangaré",
      "phone": "+2250707070707",
      "photo_url": null
    },
    "companions": [{"name": "Aïssatou Sangaré", "kind": "adult"}],
    "payment_status": "PAID",
    "options": [{"label_fr": "Transfert", "quantity": 1}]
  },
  "scan_action": "ALLOW",
  "warnings": []
}
```

Convention Idempotency-Key

Tout POST de création (reservation, payment) est protégé par Idempotency-Key (UUID v4 fourni par le client). Le serveur stocke la réponse 24h dans Redis. Un POST avec la même clé renvoie exactement la même réponse — y compris en cas de double clic ou de retry réseau.

Règles métier

Chaque produit porte ses propres règles, stockées en JSONB sur la colonne `products.attributes`. Les valeurs par défaut sont indiquées ci-dessous, mais peuvent être surchargées par produit.

6.1 — Matrice d'annulation et de remboursement

Type produit	≥ 72h avant	24h-72h avant	< 24h	No-show
Chambre	100%	50%	0%	0%
Day Pass	100%	100%	0%	0%
Sunset	100%	100%	0%	0%
Brunch	100%	100%	0%	0%
Activité	100%	100% (≥12h)	0%	0%
Corporate	Sur devis	Sur devis	Sur devis	0%
Événement	100% (≥30j)	50% (15-30j)	0%	0%
Membership	—	—	—	—

6.2 — No-show

- Un client est marqué **NO_SHOW** automatiquement à H+30 minutes après l'heure prévue d'arrivée (chambre) ou de slot (Day Pass, Sunset, activité).
- **Aucun remboursement** automatique. Possibilité pour la Direction d'accorder un **geste commercial** au cas par cas via le dashboard staff.
- **Statistiques dédiées** : taux de no-show par produit, par canal d'acquisition, par nationalité. Permet d'identifier les canaux à fort risque et d'ajuster les politiques d'acompte.
- **Score de risque client** : après 2 no-show, le client doit payer 100% à l'avance pour toute réservation future.

6.3 — Reprogrammation

- **Gratuite** si demandée ≥ 72h avant la date prévue, sous réserve de disponibilité.
- **Frais 10%** si entre 24h et 72h avant.
- **Non autorisée** à moins de 24h — le client doit annuler et re-réserver selon les règles d'annulation standard.
- **1 seule reprogrammation gratuite** par réservation. Les suivantes facturent 10% à chaque fois.

6.4 — Période de grâce de paiement

Pour les réservations chambres avec acompte 50% : le solde doit être payé **48h avant l'arrivée**. À H-48h, un rappel automatique WhatsApp + Email est envoyé. À H-24h sans paiement, la réservation est **automatiquement annulée** et l'acompte n'est pas remboursé (sauf décision Direction).

6.5 — Overbooking volontaire (chambres)

Pour absorber le no-show structurel (5-8% hôtellerie standard), le système autorise un **overbooking de 5%** sur les chambres (paramétrable). Si la situation se matérialise (toutes les chambres présentes), procédure de **relogging** dans un hôtel partenaire payé par BBr + bon d'achat de compensation.

Gestion des disponibilités

La gestion des dispos est le cœur névralgique de tout moteur de réservation. Une seule double-réservation suffit à détruire la confiance des clients. Le Booking Engine garantit l'unicité par **transactions PostgreSQL SERIALIZABLE + verrouillage ligne + contrainte d'exclusion**.

7.1 — Modèle de stock (rappel)

La table inventory stocke (**product_id, date, slot, capacity_total, capacity_sold**). Une réservation incrémente capacity_sold de quantity. L'invariant à maintenir est : $\text{capacity_sold} \leq \text{capacity_total}$.

7.2 — Algorithme atomique de réservation

```
BEGIN ISOLATION LEVEL SERIALIZABLE;

-- 1. Verrouille la ligne d'inventory
SELECT capacity_total, capacity_sold
  FROM inventory
 WHERE product_id = $1 AND date = $2 AND slot IS NOT DISTINCT FROM $3
   FOR UPDATE;

-- 2. Vérifie en mémoire applicative
IF (capacity_sold + $quantity) > capacity_total THEN
  ROLLBACK; RAISE EXCEPTION 'SOLD_OUT';
END IF;

-- 3. Incrémente
UPDATE inventory
  SET capacity_sold = capacity_sold + $quantity
 WHERE product_id = $1 AND date = $2 AND slot IS NOT DISTINCT FROM $3;

-- 4. Crée la réservation (status='PENDING' tant que paiement non validé)
INSERT INTO reservations (...) VALUES (...);

-- 5. Crée le hold (TTL 5 min)
INSERT INTO booking_holds (...) VALUES (... , now() + INTERVAL '5 minutes');

COMMIT;
```

Pourquoi SERIALIZABLE et pas READ COMMITTED ?

Avec READ COMMITTED, deux transactions concurrentes peuvent lire `capacity_sold = 249`, écrire chacune 250, et créer une double réservation. SERIALIZABLE force PostgreSQL à détecter le conflit et renvoyer une erreur que l'application retry automatiquement (jusqu'à 3 fois avant d'abandonner).

7.3 — Cache lecture

Les routes `/catalog/products` et `/catalog/availability` sont fortement cachées (Redis, TTL 30s). À chaque modification d'inventory, un invalidation event est publié et le cache est purgé instantanément pour le produit concerné.

7.4 — Gestion par type de produit

- **Chambres** — 1 chambre = 1 inventory line. Capacité = 1 par nuit. Une réservation multi-nuits réserve N lignes ($\text{jointure_date_in} + \text{date_out}$).

- **Day Pass** — 1 ligne par jour. Capacité = 250.
- **Sunset / Brunch** — 1 ligne par jour (Sunset = 120, Brunch = 200 le dimanche uniquement).
- **Activités** — 1 ligne par produit × date × slot (30/60 min). Le slot représente le créneau horaire.
- **Corporate / Événements** — 1 espace = 1 inventory line par jour. Capacité = 1 (un seul événement par espace/jour).

Gestion des paiements

Le Booking Engine intègre **6 PSPs** dès le lancement, avec une architecture **extensible** permettant d'ajouter un nouveau PSP en moins de 2 jours.

8.1 — Matrice des PSPs supportés

PSP	Type	Devise	Commission	Délai webhook	Use case
Wave	Mobile money	XOF	1.0%	< 5s	Marché CI principal
Orange Money	Mobile money	XOF	1.5%	< 10s	Marché CI/sénégal
MTN Money	Mobile money	XOF	1.5%	< 10s	CI / autres Afrique
Moov Money	Mobile money	XOF	1.5%	< 10s	CI / Bénin / Togo
Stripe	Carte	EUR/USD/XOF	2.9% + 0.30€	< 2s	Visa / Mastercard internationaux
PayPal	Wallet	EUR/USD	3.4% + 0.35€	< 5s	Touristes US / EU

8.2 — Modes de paiement

- **Paiement intégral (100%)** — option par défaut, fortement incitée par un **petit avantage** (annulation flexible jusqu'à 72h vs 48h).
- **Acompte (50%)** — disponible pour les chambres et les événements. Solde dû J-48h.
- **Acompte 30%** — réservé au Corporate (devis).
- **Paiement sur place** — uniquement avec validation Direction au cas par cas (clients VIP, partenaires de longue date). Statut pending_onsite_payment spécifique pour traçabilité.

8.3 — Séquence détaillée d'un paiement

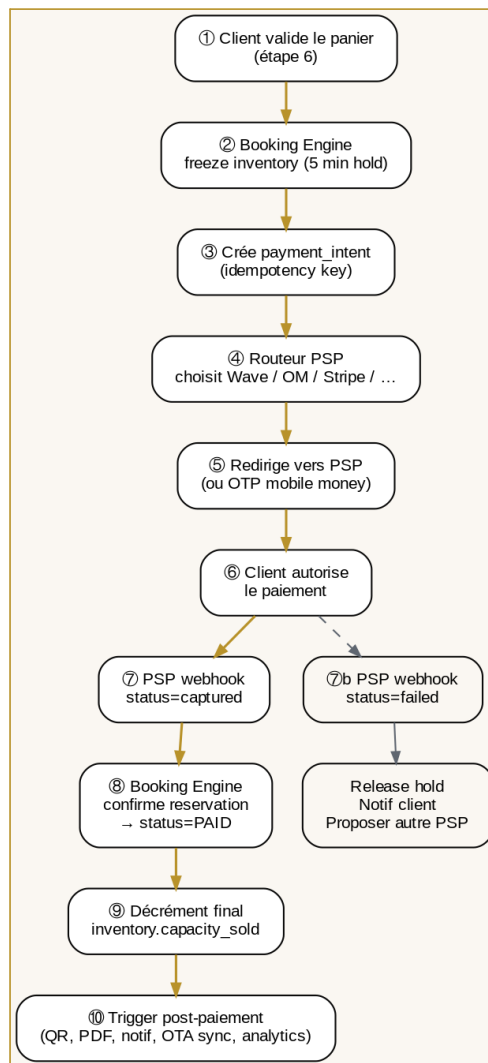


Schéma 6 — Flux complet d'un paiement (avec branche d'échec).

8.4 — Sécurité paiement

- **Webhook signature HMAC-SHA256** obligatoire — tout webhook non signé est rejeté en 401.
- **Idempotence stricte** par event_id Redis (TTL 7 jours).
- **3D-Secure 2** activé par défaut sur Stripe (cartes).
- **Tokenisation** — aucune donnée PAN stockée. Les références internes sont opaques (Stripe customer_id, Wave merchant_ref, etc.).
- **Réconciliation quotidienne** automatique entre payments et les exports des PSPs (rapport comptable matinal).

Gestion des QR Codes

Chaque réservation génère 1 à N QR codes selon le type de produit. Le QR est l'identifiant physique qui permet l'accès le jour J — sa génération, sa sécurité et sa scanabilité sont critiques.

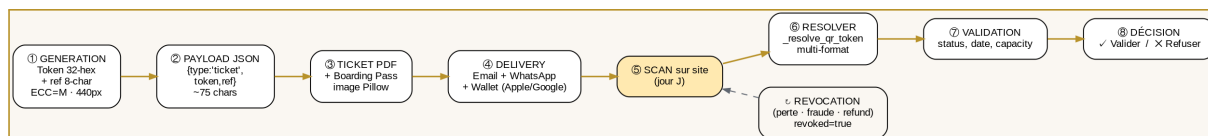


Schéma 7 — Cycle de vie complet d'un QR code.

9.1 — Génération

- **Token cryptographique** : 32 caractères hex (128 bits) générés par `secrets.token_hex(16)` — impossible à deviner ou à dupliquer.
- **Référence courte** : 8 caractères majuscules (BBR-YYYYMMDD-XXXX) — humainement lisible pour saisie manuelle en cas de scan impossible.
- **Payload compact** : JSON ~75 chars `{type:'ticket',token,ref}` — bien en dessous de la limite caméra mobile.
- **QR taille 440px, ECC=M, noir/blanc** : décodable à 50 cm de distance sur un écran de téléphone même dégradé. Validé par audit pyzbar (10/10 tests passent — voir rapport iter-48).

9.2 — Ticket PDF Premium

Le Boarding Pass délivré actuellement est **conservé tel quel** (généré via Pillow + Tickets esthétiques branded BBr). Format A4 portrait, 1080×1920 pixels, contient : logo BBR, nom du client, référence visible, produit + date + slot, QR code 440px, montant payé, conditions d'utilisation, hotline WhatsApp.

9.3 — Distribution

- **Email** : PDF en pièce jointe + miniature inline.
- **WhatsApp** : image JPG du QR + lien de téléchargement du PDF + message texte avec les détails.
- **Wallet Apple / Google** : passe wallet généré automatiquement (phase 2 du Booking Engine) avec mise à jour push si reprogrammation.
- **Espace client** : téléchargeable à tout moment.

9.4 — Scan et résolution

Le résolveur `_resolve_qr_token` accepte **5 formats** : token exact, lowercase, prefix ≥ 8 hex, JSON brut, référence courte BBR-* — déjà éprouvé dans l'app actuelle (audit complet iter-48, 0 bug trouvé).

9.5 — Révocation

Un QR peut être **révoqué** manuellement par la Direction (perte signalée par le client, suspicion de fraude, remboursement total). Le flag `revoked=true` déclenche un refus immédiat au scan avec message explicatif.

Plan d'intégration PMS + OTA

Le Booking Engine s'intègre avec deux familles de systèmes : le **PMS Front Desk / Housekeeping** (interne) en aval, et les **OTA** (Booking.com, Airbnb, Expedia, TravelOka) en amont/aval pour la distribution.

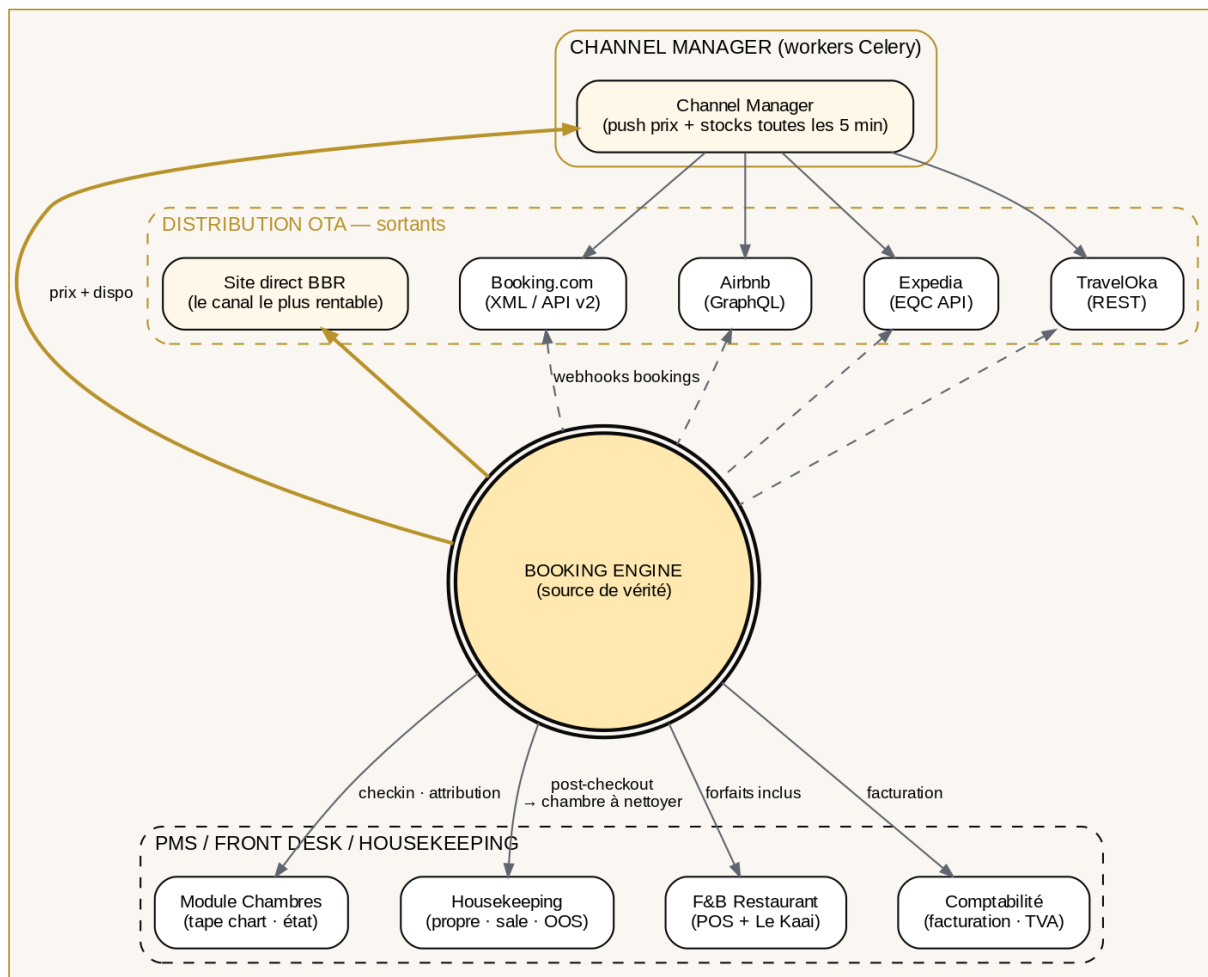


Schéma 8 — Topologie d'intégration. Le Booking Engine est la source de vérité ; le Channel Manager pousse vers les OTA et ingère les réservations entrantes.

10.1 — Channel Manager (sortants)

- **Push toutes les 5 minutes** — un worker Celery synchronise prix et disponibilités vers chaque OTA. Optimisation : ne push que les deltas (calcul du diff avec le dernier état envoyé).
- **Mapping par produit** : chaque OTA reçoit une fraction du stock total (paramétrable, ex. Booking.com = 70% des chambres, 30% gardé en direct).
- **Uplift de prix** : les commissions OTA (15-25%) sont compensées par un uplift sur le prix affiché : parité tarifaire perçue par le client mais marge préservée pour BBr.
- **Blocage exceptionnel** : bouton "stop sale" dans le dashboard staff pour fermer la vente sur tous les canaux en 30 secondes (urgence, panne, événement imprévu).

10.2 — Ingestion OTA (entrants)

Chaque OTA pousse les nouvelles réservations vers un webhook dédié (/api/booking/webhooks/ota/{platform}). Le Booking Engine : (1) crée la réservation côté PG avec channel_id = {ota}, (2) décrémente l'inventary, (3) déclenche immédiatement un re-sync vers les **autres** OTA pour fermer la dispo. Cycle complet : < 30 secondes.

10.3 — Intégration PMS (post-paiement)

- **Chambres** : à J-2 (après confirmation paiement intégral), pré-attribution automatique d'une chambre par le module Front Desk (selon préférences client + règles yield). Confirmation manuelle possible par la Réception.
- **Housekeeping** : à J+1 du check-out, déclenchement automatique du "to clean" → mobile staff.
- **F&B Restaurant Le Kaai** : si une option restaurant est incluse, le créneau est pré-réservé automatiquement.
- **Comptabilité** : facture émise automatiquement au check-in avec TVA (18% en CI), envoyée par email PDF.

10.4 — Connecteurs OTA — détail technique

OTA	Protocole	Fréquence	Endpoint
Booking.com	XML API v2 + webhooks	Push 5 min	supply.booking.com
Airbnb	GraphQL + webhooks	Push 5 min	api.airbnb.com/v2
Expedia	EQC REST + webhooks	Push 5 min	api.expediapartnersolutions.com
TravelOka	REST + polling	Push 5 min / Pull 10 min	ipi.traveloka.com
Hostelworld	iCal export	Pull horaire	hostelworld.com/ical

Espace client

Portail self-service disponible à workflow-boulaybeachresort.com/account.

Authentification

OTP via WhatsApp en priorité (Twilio Verify), email en fallback. Pas de mot de passe à mémoriser.
Session JWT 7 jours.

Fonctionnalités

- **Mes réservations** : à venir / passées, filtres par produit.
- **Mes tickets** : téléchargement PDF, ajout au Wallet.
- **Payer le solde** : un clic, choix du PSP.
- **Modifier mes informations** : téléphone, email, nationalité (lecture seule sur nom + prénom une fois vérifiés).
- **Mon historique** : chiffrement RGPD, anonymisation possible sur demande.
- **Mes points fidélité** : solde, historique, récompenses débloquées.
- **Reprogrammer / annuler** : self-service respectant les règles métier (§6).

Dashboard interne

Dashboard staff disponible à /staff/booking. Réservé aux rôles Réception, Comptabilité, Direction, Marketing.

Widgets principaux

- **Réservations** : vues aujourd'hui / semaine / mois / personnalisée, avec filtres canal, produit, statut. Drill-down 1-clic sur chaque ligne.
- **Chiffre d'affaires** : total + ventilation par produit, catégorie, canal, campagne. Comparatif année-1 / mois-1.
- **Occupation** : chambres (tape chart), activités (grille slots), événements (calendrier des espaces).
- **Paiements** : encaissés / en attente / remboursés. Réconciliation en un clic avec les exports PSPs.
- **Alertes** : solde dû à 24h, no-show probable, double-booking détecté, panne PSP, OTA en retard de sync.

Contrôle d'accès — scanner QR

Le module Contrôle d'accès (déjà en place dans l'app actuelle) est conservé. Accessible à /staff/scan pour les rôles Réception et Contrôle Embarquement.

Affichage après scan

- **Photo du client** (si disponible — sinon initiales).
- **Statut paiement** : PAID / PARTIAL / PENDING.
- **Produit réservé** avec date et slot.
- **Nombre d'accompagnants** + leurs noms.
- **Options additionnelles** : transfer, sunset, etc.
- **Warnings éventuels** : première visite, membre Platinum (à accueillir avec attention), allergie déclarée.

Actions disponibles

✓ **Valider accès** (par défaut, incrémente scan_count) — ✗ **Refuser accès** (avec motif requis, génère un incident) — ■ **Historique des scans** (qui a scanné quoi, quand).

KPIs et analytics par réservation

Chaque réservation enrichit les agrégats analytics avec ces champs (capturés au moment de la création) :

- **Source d'acquisition** : direct, organic, paid, OTA-{platform}, referral, email, whatsapp.
- **Campagne publicitaire** (FK campaigns) si attribuée via cookie UTM ou Conversion API Meta.
- **UTM source / medium / campaign / term / content**.
- **Device** : mobile / tablet / desktop. **OS** et **navigateur**.
- **Géolocalisation** : ville et pays via lookup IP (MaxMind ou Cloudflare GeoIP).
- **Canal de réservation** : direct site, web app, espace client, staff manuel, OTA.
- **Temps de session avant booking** : indicateur du temps d'hésitation (mesure d'efficacité du tunnel).

KPIs hebdomadaires automatiques

KPI	Formule	Objectif
Taux de conversion tunnel	bookings / sessions	≥ 32%
Panier moyen	$\Sigma \text{ amount_total} / N \text{ bookings}$	+18% vs 2025
RevPAR (chambres)	CA chambres / nuits dispo	75 000 XOF
Taux de no-show	no_show / paid	< 5%
ROAS Meta	CA Meta / dépense Meta	≥ 6x
ROAS Google Ads	CA Gads / dépense Gads	≥ 8x
Coût d'acquisition (CAC)	$\Sigma \text{ dépenses ads} / N \text{ nouveaux}$	< 8 000 XOF
LTV / CAC	LTV moyen / CAC	≥ 5x
Délai de confirmation	p95 (paid → ticket reçu)	< 4s
Délai de réponse PSP	p95 webhook	< 10s

Fin du document

Ce document est **vivant** et sera mis à jour à chaque phase majeure. La prochaine étape est la **conception détaillée des wireframes** (vrais mockups Figma) et la **spécification du Channel Manager OTA**. À venir dans le PROMPT 3.